

PROJECTE HOSPITAL DE BLANES

Hospital Santa Paciencia

DOCUMENT FINAL



Josep Verdalet & Àlex Planas

INDEX

CONNECTIVITAT I LOGIN.....	3
MATRIU DE SEURETAT.....	7
Assignació d'Usuaris.....	8
DATA MASKING.....	8
SSL.....	9
BLOC DE MANTENIMENT.....	12
BLOC ALTA DISPONIBILITAT.....	14
1. Node Actiu (Servidor de Producció - Hospital).....	14
2. Node Passiu (Hot Standby - Cloud/Remot).....	15
BLOC DE CONSULTES.....	16
DUMMY DATA.....	18
1. Objectiu del Dummy Data en el Projecte.....	18
2. Funcionament Tècnic de l'Script (Pas a Pas).....	18
POWER BI.....	21

CONNECTIVITAT I LOGIN

readme → [enllaç](#)

Aquest bloc forma part de l'aplicació de gestió de centres mèdics "**Sistema de Gestió Hospitalària**", i implementa de manera centralitzada tot el sistema d'inici de sessió i registre d'usuaris amb validació, seguretat avançada de credencials, connexió a base de dades PostgreSQL i una interfície gràfica d'usuari (GUI) amb Tkinter.



Característiques i Funcionalitats Implementades

- **Interfície gràfica amb Tkinter:** Disseny d'una GUI íntegra per a la interacció amb l'usuari durant l'inici de sessió i el registre.
- **Canvi de mode dinàmic:** Transició fluida entre el mode "Login" i "Registre" dins de la mateixa finestra de l'aplicació.
- **Validació de formularis:** Control de camps obligatoris, verificació de coincidència de contrasenyes i gestió d'errors gràfics amb missatges clars de confirmació o denegació.
- **Seguretat Avançada d'Accés:** Protecció robusta tant en l'emmagatzematge local de credencials com en el tractament de dades en memòria.
- **Modularitat del codi:** Divisió estructurada en múltiples fitxers (mòduls) per optimitzar el manteniment, la neteja i l'escalabilitat del programari.
- **Connexió a Base de Dades:** Integració de connexions amb PostgreSQL mitjançant la llibreria psycopg2.



Mecanismes de Seguretat Avançada

El sistema prioritza la confidencialitat de les credencials dels usuaris aplicant un doble factor de protecció en el tractament de la informació:

1. Hashing de Contrasenyes (bcrypt)

Per evitar que les contrasenyes es gestionin o s'emmagatzemin en text pla, s'utilitza la llibreria bcrypt. Aquest algorisme realitza una funció de hash robusta que incorpora de forma nativa un *salt* aleatori i un cost de computació configurable, protegint el sistema contra atacs de força bruta i taules de hash precalculades (rainbow tables).

2. Xifrat de Fitxers Locals (cryptography - Fernet)

Les dades d'accés de l'usuari es guarden en un fitxer físic separat anomenat `access.dat`. Per garantir la màxima seguretat en local:

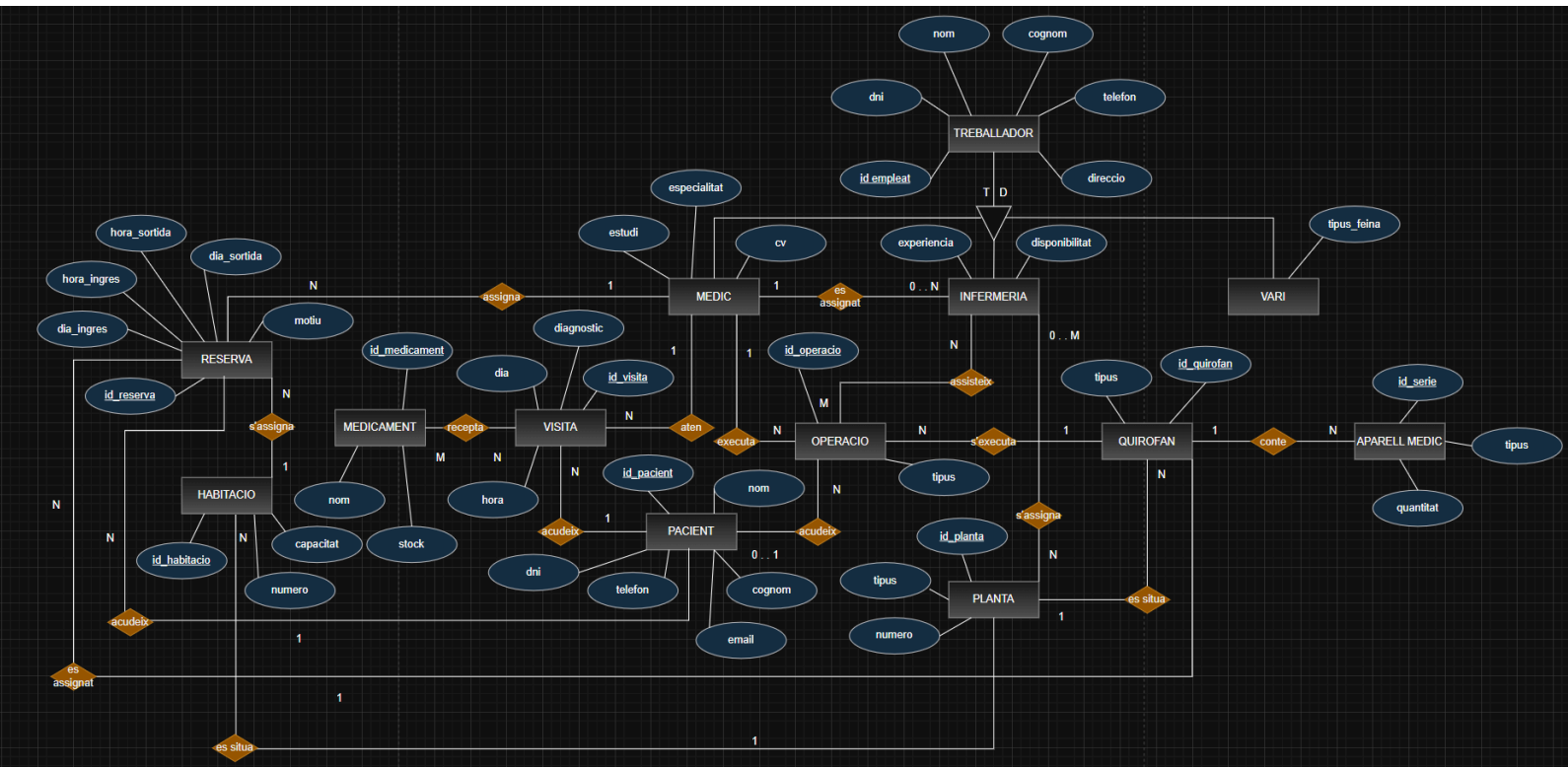
- El fitxer s'encrypta simètricament mitjançant l'estàndard **Fernet** (de la llibreria `cryptography`).
- El sistema genera i utilitza una clau secreta única (`secret.key`) per dur a terme el procés de xifrat i desxifrat.
- Aquesta arquitectura protegeix tant l'identificador d'usuari com la contrasenya davant de qualsevol consulta externa no autoritzada al fitxer de configuració.

Estructura del Projecte

L'arquitectura del codi font s'organitza de manera modular en els següents fitxers:

- **main.py**: Punt d'entrada de l'aplicació. Gestiona l'entorn de la finestra de Tkinter, el posicionament dels camps de text de la interfície i la programació de la lògica dels botons de login i registre.
- **seguretat.py**: Centralitza tota la lògica criptogràfica. Conté les funcions encarregades de generar les claus secretes, xifrar o desxifrar el fitxer de dades local (`access.dat`) i verificar el hash de les contrasenyes.
- **database.py**: Gestiona exclusivament la infraestructura de connexió. Centralitza els paràmetres d'accés al servidor PostgreSQL (com ara `host`, base de dades, usuari i contrasenya).
- **requirements.txt**: Fitxer de text on es recullen totes les llibreries externes imprescindibles perquè el projecte pugui ser instal·lat i executat amb èxit en qualsevol màquina.

MODEL ER-RELACIONAL



atribut → minúscula / color blau

clau primària → minúscula subratllat / color blau

ENTITAT → majúscula / color negre

relació → minúscula / color taronja

Relacional → [enllaç](#)

MATRIU DE SEGURETAT

Rol d'Usuari	Accés a Taules / Vistes	Operacions Permeses
dba	Totes les taules, seqüències i funcions de l'esquema hospital	SELECT, INSERT, UPDATE, DELETE, CREATE, DROP (ALL PRIVILEGES)
administracio	PACIENT, RESERVA, HABITACIO, PLANTA, TREBALLADOR	SELECT, INSERT, UPDATE (Nota: A TREBALLADOR només SELECT)
medic	VISITA, RECEPTE, OPERACIO, PACIENT, MEDICAMENT, INFERMERIA	SELECT, INSERT, UPDATE (Nota: A PACIENT, MEDICAMENT i INFERMERIA només SELECT)
infermeria	ASSISTEIX, OPERACIO, VISITA, PACIENT, RESERVA, MEDICAMENT	SELECT, INSERT (Nota: Només té permís d'INSERT a la taula ASSISTEIX)
vari	APARELL_MEDIC, MEDICAMENT, QUIROFAN, PLANTA, v_pacient_vari	SELECT, INSERT, UPDATE (Nota: A QUIROFAN, PLANTA i a la vista anònima v_pacient_vari només SELECT)
zelador	RESERVA, HABITACIO, OPERACIO, QUIROFAN, APARELL_MEDIC, PLANTA, v_pacient_zelador	SELECT

El disseny de l'script està pensat per a un entorn d'hospital, on s'aplica el principi de *mínim privilegi*, cada rol té accés només a les taules que necessita per a la seva feina.

[enllaç](#)

Assignació d'Usuaris

Finalment, l'script crea un usuari d'exemple per a cada rol, li assigna una contrasenya temporal i li atorga el rol corresponent (heretant així tots els permisos i restriccions esmentats anteriorment):

- u_dba_exemple → Rol dba
- u_administracio_exemple → Rol administracio
- u_medic_exemple → Rol medic
- u_infermeria_exemple → Rol infermeria
- u_vari_exemple → Rol vari
- u_zelador_exemple → Rol zelador

DATA MASKING

[enllaç](#)

L'script crea dues vistes idèntiques (v_pacient_vari i v_pacient_zelador) per als rols vari i zelador. La restricció s'aplica mitjançant funcions de text per **anonimitzar dades sensibles**:

- **DNI:** Es canvien els primers caràcters per 'XXXXX' i només es veuen els 4 últims (Ex: XXXXX1234A).
- **Telèfon:** Es canvien els primers per asteriscs i només es veuen els 3 últims (Ex: *****789).
- **Email:** Es manté la primera lletra i el domini, amagant la resta de l'usuari (Ex: j*****@gmail.com).

SSL

[enllaç](#)

1. Creació del directori i instal·lació de dependències

Primer s'actualitza el sistema, s'instal·la l'eina openssl i es crea el directori on s'emmagatzemaran els certificats assignant-li els permisos adequats a l'usuari postgres:

```
sudo apt update
sudo apt install openssl -y
sudo mkdir -p /etc/postgresql/ssl
sudo chown postgres:postgres /etc/postgresql/ssl
sudo chmod 700 /etc/postgresql/ssl
```

2. Generació del Certificat i la Clau Privada

Es genera un certificat auto-signat vàlid per a 365 dies amb una clau RSA de 4096 bits utilitzant la IP del servidor:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:4096 \
    -keyout /etc/postgresql/ssl/server.key \
    -out /etc/postgresql/ssl/server.crt \
    -subj "/CN=192.168.56.103"
```

3. Assignació de Permisos als Fitxers generats

Es transfereix la propietat dels fitxers a l'usuari postgres i es restringeixen els permisos perquè només el propietari pugui llegir la clau privada (requisit obligatori de PostgreSQL):

```
sudo chown postgres:postgres /etc/postgresql/ssl/server.key
/etc/postgresql/ssl/server.crt
sudo chmod 600 /etc/postgresql/ssl/server.key
sudo chmod 644 /etc/postgresql/ssl/server.crt
```

4. Configuració de PostgreSQL

S'edita el fitxer de configuració principal postgresql.conf per activar el paràmetre SSL i indicar les rutes dels fitxers acabats de crear:

```
sudo nano /etc/postgresql/16/main/postgresql.conf
```

Dins del fitxer s'han de modificar o afegir les línies següents:

```
ssl = on
ssl_cert_file = '/etc/postgresql/ssl/server.crt'
ssl_key_file = '/etc/postgresql/ssl/server.key'
```

Per aplicar els canvis es reinicia el servei de base de dades:

```
sudo systemctl restart postgresql
```

5. Automatització de la Renovació del Certificat

Es crea un script de Bash anomenat renovar-postgres-ssl.sh situat a /usr/local/bin/ per automatitzar el procés de renovació abans que expiri.

Contingut bàsic de l'script (/usr/local/bin/renovar-postgres-ssl.sh):

```
#!/bin/bash
DIES=365
RUTA="/etc/postgresql/ssl"
IP_VM="192.168.56.103"

openssl req -x509 -nodes -days $DIES -newkey rsa:4096 \
    -keyout $RUTA/server.key \
    -out $RUTA/server.crt \
    -subj "/CN=$IP_VM"

chown postgres:postgres $RUTA/server.key $RUTA/server.crt
chmod 600 $RUTA/server.key
chmod 644 $RUTA/server.crt
systemctl restart postgresql
```

Es donen permisos d'execució a l'script:

```
sudo chmod +x /usr/local/bin/renovar-postgres-ssl.sh
```

I s'afegeix al **crontab** del sistema (sudo crontab -e) per programar la seva execució automàtica (guardant el log de sortida):

```
00 11 * * * /usr/local/bin/renovar-postgres-ssl.sh >>  
/var/log/renovacio-ssl-p
```

6. Verificació del funcionament

Finalment, es connecta a la consola de PostgreSQL per comprovar que el paràmetre SSL està correctament activat:

```
sudo -u postgres psql
```

I dins de la consola s'executa:

```
SHOW ssl;
```

BLOC DE MANTENIMENT

[enllaç](#)

El mòdul de manteniment de l'**Hospital Santa Paciencia** s'ha desenvolupat amb **Tkinter** per gestionar l'activitat diària de forma organitzada i modular, estructurant el codi en 4 scripts per optimitzar-ne el manteniment:

Característiques Principals

- **Gestió i Manteniment:** Altes i baixes de personal mèdic, infermeria, pacients i assignació de recursos.
- **Consultes i Informes:** Monitorització d'ocupació de plantes, operacions diàries i històrics de visites.

Estructura dels Fitxers

1. main.py (Fitxer Principal)

- Aixeca la finestra inicial de *Login* (usuari i contrasenya).
- Si les credencials són correctes i hi ha connexió, amaga la finestra amb `root.withdraw()` i obre el menú de manteniment.
- Permet el registre de nous usuaris.

2. seguretat.py (Xifratge i Privadesa)

- Utilitza **Fernet** (cryptography) per generar una clau simètrica (`secret.key`) i xifrar el fitxer d'accessos local (`access.dat`).
- Aplica un hash amb **bcrypt** a les contrasenyes abans de desar-les al JSON xifrat (evitant el text pla).

3. database.py (Connexió a la Base de Dades)

- Gestiona la connexió amb PostgreSQL mitjançant **psycopg2**.
- Configura `sslmode="require"` per garantir la seguretat a través del certificat SSL del servidor.
- Força el `search_path` cap a l'esquema `hospital` per simplificar les consultes del codi.

4. gui_manteniment.py (Interfície Gràfica)

Finestra principal de gestió dividida en dues seccions:

- **Manteniment:** Gestió d'altres de personal (crida la funció `hospital.alta_treballador_hospital` de PostgreSQL i agafa l'`ID SERIAL`), altes de pacients, consultes de planta d'infermeria, operacions del dia i eliminacions.
- **Consultes i Informes:** Llistats analítics com el recompte de recursos per planta (`COUNT`), informe de rols amb `CASE WHEN` (Metges, Infermers o Variat) i visites totals.

Procediments i Consultes SQL

Nom / Eina	Tipus	Descripció
hospital.alta_treballador_hospital	Funció PL/pgSQL	Insereix el personal i retorna l'ID generat pel SERIAL.
COUNT	Agregació SQL	Recompte dels recursos d'una planta de l'hospital.
CASE WHEN	Lògica SQL	Classifica el personal segons el seu rol (Metge, Infermer, Variat).

BLOC ALTA DISPONIBILITAT

[enllaç](#)

Assegurar la continuïtat assistencial de l'hospital **24x7**, evitant qualsevol interrupció en l'accés a l'historial clínic o la gestió de visites. Es proposa una arquitectura de **dos nodes en replicació física** per suportar una càrrega de més de **100.000 visites** i garantir la seguretat de les dades confidencials segons l'AGPD.

1. Node Actiu (Servidor de Producció - Hospital)

Aquest servidor estarà físicament al datacenter de l'hospital per minimitzar la latència en les consultes de metges i infermeres.

Component	Detalls Tècnics	Preu Estimat
CPU	AMD EPYC 7232P (8 Cores / 16 Threads)	480 €
Memòria RAM	128 GB DDR4 ECC	350 €
Emmagatzematge	2x 1.92TB NVMe Enterprise	650 €
Connexió	1Gbps fibra	45€/mes
Sistema Operatiu	Ubuntu Server 24.04 LTS	0 €
Base de Dades	PostgreSQL 16 amb suport UTF-8 (Ciríl·lic)	0 €

Justificació Tècnica:

- **Memòria Elevada:** Es trien 128 GB de RAM per permetre que tota la base de dades activa i els seus índexs (especialment els de caràcters ciríl·lics) resideixin en memòria cache, accelerant les consultes.
- **Volum de Dades:** Dissenyat per processar sense retards les **50.000 fitxes de pacients** i els accessos simultanis dels **450 empleats** del centre.

2. Node Passiu (Hot Standby - Cloud/Remot)

Ubicat fora de l'hospital per actuar com a node de recuperació davant desastres físics (incendis, inundacions o talls elèctrics prolongats).

Component	Detalls Tècnics	Preu Estimat
CPU	Instància Cloud Equivalent (8 vCPUs)	45 €/mes
Memòria RAM	64 GB RAM	Inclòs
Emmagatzematge	2 TB SSD Provisioned IOPS	Inclòs
Connexió	Enllaç xifrat via SSL/TLS obligatori	0 €
Estat	<i>Hot Standby</i> (Llegible per a informes)	0 €

Justificació Tècnica:

- **Replicació Asíncrona:** El node cloud rep els canvis en temps real des del node principal.
- **Reporting:** Tot i ser un node passiu per a escriptures, es pot utilitzar per generar els informes XML massius per a la Seguretat Social sense carregar el servidor principal.
- **SSL:** Tota la replicació es fa sota túnels SSL per complir amb el nivell alt de seguretat demanat.

Cost Total de Hardware Base: ~1.600 €

BLOC DE CONSULTES

[enllaç](#)

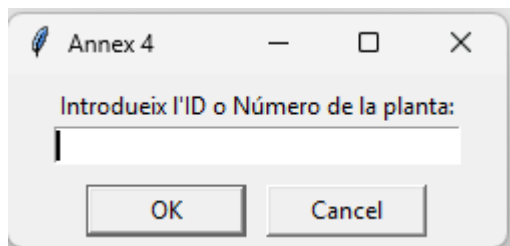
El bloc de consultes hem fet servir el fitxer gui_manteniment.py.

Les consultes que es poden fer són aquestes quatre:

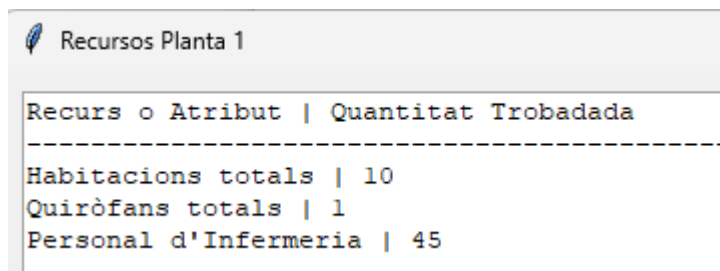
1. **Informe General de Personal:** Llista tot el personal de l'hospital i mostra de forma dinàmica la seva dada extra segons el rol (especialitat, experiència o tipus de feina).
2. **Informe de Visites per Dia:** Un comptador estadístic que mostra el volum total de visites agrupades per cada data.
3. **Recursos per Planta:** Mostra una radiografia de la infraestructura d'una planta (total d'habitacions, quiròfans i infermers assignats).

Recursos per planta

Permet veure quina quantitat hi ha per a cada planta



A dialog box titled 'Annex 4' with a feather icon. It contains a text input field with the placeholder 'Introdueix l'ID o Número de la planta:'. Below the input field are two buttons: 'OK' and 'Cancel'.



A window titled 'Recursos Planta 1' with a feather icon. It displays a table with the following content:

Recurs o Atribut	Quantitat Trobadada
Habitacions totals	10
Quiròfans totals	1
Personal d'Infermeria	45

Personal Hospital

Mostra un llistat complet de tots els metges i infermers i personal vari que tenim en plantilla

Informe General de Personal						
ID	DNI	Nom	Cognom	Telèfon	Rol / Categoria	
1	10000001S	Guiomar	Solana	+34741667610	MÈDIC (Ginecologia)	
2	10000002Q	Maribel	Ramirez	+34981538688	MÈDIC (Medicina General)	
3	10000003V	Almudena	Bernat	+34836 33 51 80	MÈDIC (Ginecologia)	
4	10000004H	Saturnina	Agustí	+34900 932 605	MÈDIC (Neurologia)	
5	10000005L	Marina	Morán	+34 871 992 381	MÈDIC (Pediatria)	
6	10000006C	Celestina	Escudero	+34 902 68 19 0	MÈDIC (Urgències)	
7	10000007K	Zacarias	Barranco	+34986584432	MÈDIC (Urgències)	
8	10000008E	Lidia	Villalobos	+34932800328	MÈDIC (Medicina General)	
9	10000009T	Feliciano	Alsina	+34944722491	MÈDIC (Ginecologia)	
10	10000010R	Hernando	Sedano	+34980 59 76 68	MÈDIC (Pediatria)	
11	10000011W	Diana	Salinas	+34922 75 55 75	MÈDIC (Medicina General)	
12	10000012A	Jordana	Conesa	+34 975698123	MÈDIC (Pediatria)	
13	10000013G	Hipólito	Cerdán	+34 822761642	MÈDIC (Pediatria)	

Visites Per Dia

veiem el número total de visites q tenim per dia

Informe: Visites totals per Dia	
Data (Dia)	Nombre de Visites Registrades
2026-05-19	308
2026-05-18	288
2026-05-17	255
2026-05-16	304
2026-05-15	272
2026-05-14	307
2026-05-13	267
2026-05-12	260
2026-05-11	278
2026-05-10	263
2026-05-09	262
2026-05-08	264
2026-05-07	281

DUMMY DATA

script → [enllaç](#)

1. Objectiu del Dummy Data en el Projecte

Segons els requeriments de l'Hospital, el centre necessita fer proves inicials de rendiment de la base de dades utilitzant un volum massiu de dades falses (*fake*), però que tinguin **sentit lògic, consistència i un format correcte** (com DNI vàlids).

L'script compleix exactament amb les quotes exigides en el plec del projecte:

- **50.000** Pacients.
- **100.000** Visites mèdiques.
- **400** Treballadors en total (repartits en 100 metges, 200 infermeres i 100 de personal vari [personal de neteja i administració]) .

2. Funcionament Tècnic de l'Script (Pas a Pas)

L'script s'estructura com una funció principal anomenada `ejecutar_generacio_directa()`, la qual executa les següents fases seqüencials utilitzant la llibreria `psycpg2` per interactuar amb PostgreSQL i `Faker` per generar dades aleatòries realistes:

Connexió i Configuració Inicial

- Es connecta al servidor de la base de dades PostgreSQL allotjat a la IP indicada (192.168.56.103) fent ús de les credencials administratives.
- Força el canvi de context de treball cap a l'esquema específic de l'hospital executant `SET search_path TO hospital;`

Neteja en Cascada (`TRUNCATE ... CASCADE`)

- Abans d'inserir res, l'script buida completament 17 taules del model relacional en un ordre lògic (des de les taules de relacions com `RECEPTE` o `ASSISTEIX` fins a les entitats mestres com `PACIENT` o `TREBALLADOR`).
- S'utilitza `RESTART IDENTITY CASCADE` per reiniciar els comptadors autoincrementals (serials) i evitar errors de claus foranes (FK).

Generació de l'Estructura de Personal i Hospital

La inserció massiva es realitza mitjançant `execute_values`, una eina de `psycpg2` molt més ràpida que els `INSERT` individuals, ja que agrupa les dades en una sola sentència SQL.

TREBALLADOR: Genera 400 empleats bàsics i amb Faker afegeix noms, cognoms, telèfons i adreces realistes en espanyol.

Especialitzacions del Personal: Divideix els 400 IDs generats en tres subgrups per heretar o omplir les taules de rol:

- **MEDIC (1 al 100):** Se'ls assigna una especialitat mèdica aleatòria (Pediatría, Cardiologia, etc.) , universitat i un CV fictici.
- **INFERMERIA (101 al 300):** Se'ls assigna de forma aleatòria l'experiència (1-25 anys) i el torn de treball (Matí, Tarda, Nit).
- **VARI (301 al 400):** Se'ls assigna el tipus de feina (Neteja, Administració, Manteniment, Seguretat).

Infraestructura (PLANTA, HABITACIO, QUIROFAN, APARELL_MEDIC):
Es creen 5 plantes , 10 habitacions per planta (amb capacitat aleatòria d'1, 2 o 4 llits), 5 quiròfans i 20 aparells mèdics (com respiradors o desfibril·ladors) distribuïts en aquests quiròfans.

MEDICAMENTOS: S'introdueix un catàleg inicial de 7 medicaments comuns amb estocs aleatoris (entre 10 i 500 unitats).

D. Generació de Volum Massiu de Proves

Aquí és on s'injecta la càrrega crítica de dades per avaluar el rendiment de la base de dades:

- **PACIENT (50.000 registres):** Es generen 50.000 pacients únics amb el seu propi DNI correcte, nom, cognoms, email i telèfon ficticis.
- **VISITA (100.000 registres):** Es creen 100.000 visites mèdiques enllaçant aleatòriament els pacients i els metges existents.

E. Operacions, Reserves i Relacions Finals

- **OPERACIO:** Crea 500 intervencions quirúrgiques distribuïdes entre els 5 quiròfans, pacients i metges.
- **RESERVA:** Genera 1.000 ingressos hospitalaris de pacients.
- **RECEPTE i ASSIGNA:** Vincula aleatòriament 5.000 receptes de medicaments a les visites, i assigna les 200 infermeres a les 5 plantes de l'hospital.

F. Transacció i Tancament (commit)

- Finalment, si tot el procés s'ha executat sense cap fallida, es fa un `conn.commit()` que desa permanentment totes les dades a la base de dades PostgreSQL de l'Ubuntu Server.
- Si hi hagués qualsevol error durant les insercions, s'executa un `conn.rollback()` per deixar la base de dades intacta (com estava abans de començar), garantint la **atomicitat** de la inserció. Al final, es tanquen correctament els cursors i les connexions.

POWER BI

[enllaç](#)

Power BI és una eina de Microsoft que recull dades de diferents fonts per transformar-les en gràfics interactius i panells visuals fàcils d'entendre en temps real.

El gràfic ens servirà per veure quins serveis estan més saturats i quins estan més tranquils. Això ajuda l'hospital a saber on hi ha més volum de feina per enviar-hi més personal de reforç i evitar que els pacients hagin d'esperar tant.

